

Junior Astronaut Mission Trainer

Team Role Guide - NASA Space Apps Challenge 2026

Project in one line

An interactive simulation where students experience what it takes to manage a Moon/Mars outpost by balancing limited resources and responding to unexpected mission events.

Core idea

NASA science -> simulation -> decision -> consequence -> learning. The identity of the project is an **interactive space mission simulation**. AI is a supporting Mission Advisor, not the main selling point.

Key dates

- NASA Space Apps hackathon: **November 14-15, 2026**
- Submission window: Nov 14, 9:00 AM to Nov 15, 11:59 PM local time
- Team internal deadline: **November 13** (working MVP ready before the event)

MVP scope

1 planet, 5 resources (Power, Water, Food, Life Support, Radiation Protection), 10-15 events, scoring, AI advisor, NASA references. Moon + Mars, multiplayer, 3D and leaderboards only if time remains.

Roles ranked by difficulty

Rank	Role	Difficulty	Why
1	NASA Research & Content	Easy-Medium	Research, sources, scenarios, explanations; very little coding
2	Frontend / UI-UX	Medium	React/Vite, dashboard, responsive UI, animations
3	Game Logic / Simulation	Medium-Hard	Resources, events, decisions, scoring, state management
4	AI / Backend & Integration	Hardest	AI + APIs + connecting everyone's work + deployment/testing

Note: difficulty depends on each person's skills. Someone strong in React may find Frontend easy; someone strong in Python/APIs may find the AI role easier than Game Logic. Assign roles by existing skill, not just by difficulty.

1. NASA Research & Content

Difficulty: Easy-Medium

Main mission

Make the game scientifically grounded. Every event, number and explanation must be traceable to a real source.

Key tasks

- Find NASA sources on Moon/Mars habitats, power, water recycling, radiation, life support
- Write 10-15 mission events (solar power drop, water recycling failure, radiation storm, comms failure, equipment breakdown, medical emergency, etc.)
- For each event write: scientific fact, NASA source, why it matters, how it affects gameplay
- Write the educational feedback shown after each decision (WHY + NASA Connection)
- Maintain the reference list; NASA terms require teams to cite sources used
- Later: build the pitch script and help with the presentation

Deliverables

- Event content sheet (shared with Game Logic)
- Reference/citation document
- Educational explanation texts
- Fact-check report

Timeline

Period	Focus
Sep 29 - Oct 5	NASA research, data sources
Oct 6 - 12	Scientific scenarios for every event
Oct 13 - 19	Content ready for the MVP
Oct 20 - 26	References and citations
Oct 27 - Nov 2	Full educational content
Nov 3 - 7	Fact checking
Nov 8 - 10	Science testing (numbers, terms)
Nov 11 - 13	Final references
Nov 14 - 15	Presentation

Biggest challenge

Keeping content accurate while making it simple enough for students.

Tip

Agree on one spreadsheet/JSON format with the Game Logic member early so content drops straight into the game.

2. Frontend / UI-UX

Difficulty: Medium

Main mission

Build everything the player sees and touches, and show the game engine's data clearly.

Key tasks

- Landing page and mission selection screen
- Main mission dashboard with resource bars, day counter and score
- Event card and decision buttons
- Result screen with educational feedback
- Final mission report screen
- Animations, transitions, icons, responsive/mobile design

Deliverables

- React/Vite app with all 6-8 screens
- Reusable components (resource bar, event card)
- Responsive, polished UI

Timeline

Period	Focus
Sep 29 - Oct 5	UI wireframes
Oct 6 - 12	Dashboard design
Oct 13 - 19	React MVP (playable but ugly)
Oct 20 - 26	Integration with engine
Oct 27 - Nov 2	Full UI
Nov 3 - 7	Polish and animations
Nov 8 - 10	UI testing on desktop and mobile
Nov 11 - 13	Final UI
Nov 14 - 15	Final fixes

Biggest challenge

Displaying Game Logic data correctly in the UI and keeping both sides in sync.

Tip

Agree on the mission state object with Game Logic on day one and build the UI against mock data until the engine is ready.

3. Game Logic / Simulation

Difficulty: Medium-Hard

Main mission

Build the engine that runs the mission: resources, events, decisions, consequences and scoring.

Key tasks

- Design the state object: power, water, food, life support, radiation, score, mission day
- Implement the loop: event -> decision -> state update -> score update -> next event
- Define 3 decisions per event with resource effects (e.g. Repair: Power -15, Water +10)
- Scoring rules and win/lose conditions
- Prevent bugs: negative resources, broken score, repeating events
- Balance difficulty over a 30-day mission

Deliverables

- Game engine module (independent of UI)
- Events and decisions data file
- Scoring and win/lose rules
- Balance notes

Timeline

Period	Focus
Sep 29 - Oct 5	Game mechanics
Oct 6 - 12	Events + scoring design
Oct 13 - 19	Working game engine
Oct 20 - 26	Integration
Oct 27 - Nov 2	Full gameplay
Nov 3 - 7	Balancing
Nov 8 - 10	Game testing
Nov 11 - 13	Final mechanics
Nov 14 - 15	Final fixes

Biggest challenge

Deceptively hard: with many events, state management and balancing get complicated.

Tip

Keep the engine as pure functions (state + decision = new state) so it is easy to test and easy for the UI to use.

4. AI / Backend & Integration

Difficulty: Hardest

Main mission

Build the AI Mission Advisor and act as the integration owner who makes everyone's work run together.

Key tasks

- Set up the shared GitHub repo, structure and branching rules
- AI Mission Advisor: prompt design, API, safe answers only about the mission
- Fallback responses when the API fails; handle empty or irrelevant questions
- Backend/API if required, environment variables, secrets
- Connect Frontend, Game Engine, data and AI
- Deployment, testing, README, demo video and final submission

Deliverables

- Working AI advisor with fallback
- Integrated, deployed app
- Complete README (run guide, tech, architecture, NASA data, AI usage, team, license)
- Submission on Nov 14-15

Timeline

Period	Focus
Sep 29 - Oct 5	Architecture, repo setup
Oct 6 - 12	AI design
Oct 13 - 19	API/backend
Oct 20 - 26	AI integration
Oct 27 - Nov 2	Full system
Nov 3 - 7	AI polish, better prompts
Nov 8 - 10	API testing
Nov 11 - 13	Deployment
Nov 14 - 15	Integration + submission

Biggest challenge

Responsible for the whole system working together, not just one feature.

Tip

Integrate early and often (small merges every few days) instead of connecting everything at the end. The game must still work if the AI fails.

Team Working Rules

- One shared GitHub repo; do not work independently until the last moment
- Daily short check-in on Discord/Meet and a shared task board (virtual Universal Event workflow)
- Oct 19: playable but ugly version. Nov 2: full MVP. Nov 13: internal deadline
- Next step: write the Game Design Document (5 resources, 15 events, 3 decisions each, scoring, win/lose, NASA source structure)